

三、并发与多线程

1. 线程的生命周期与状态

1.1 线程状态

1. Java线程有哪几种基本状态？它们之间是如何转换的？ ★★★★★☆

Java线程在其生命周期中，总共包含 6 种状态，定义在 `java.lang.Thread.State` 枚举中。

- 1. **NEW (新建)**: 线程对象被创建，但尚未调用 `start()` 方法。
- 2. **RUNNABLE (可运行)**: 这是Java中一个复合状态。
 - **Ready (就绪)**: 调用 `start()` 方法后，线程进入就绪状态，等待CPU调度。
 - **Running (运行)**: 获取到CPU时间片，正在执行代码。
- 3. **BLOCKED (阻塞)**: 线程等待进入一个 `synchronized` 同步代码块或方法，但该锁被其他线程持有。
- 4. **WAITING (无限期待)**: 线程等待某个特定条件被满足。这通常由调用 `Object.wait()` , `Thread.join()` , 或 `LockSupport.park()` 引起。它需要被其他线程显式地唤醒。
- 5. **TIMED_WAITING (限时等待)**: 与 `WAITING` 类似，但有一个指定的等待时间。超时后会自动唤醒。由调用 `Thread.sleep()` , `Object.wait(timeout)` , `Thread.join(timeout)` 等方法引起。
- 6. **TERMINATED (终止)**: 线程的 `run()` 方法执行完毕或因异常退出。

状态转换图:



2. 线程在哪些情况下会进入BLOCKED、WAITING和TIMED_WAITING状态？ ★★★★★☆

- **进入 BLOCKED (阻塞) 状态:**
 - 唯一情况：线程试图进入一个被其他线程持有的 `synchronized` 代码块或方法时。
- **进入 WAITING (无限期待) 状态:**
 - 1. 调用 `Object.wait()` (无参)，线程释放锁并进入等待，直到被 `notify()` 或 `notifyAll()` 唤醒。

2. 调用 `Thread.join()` (无参), 当前线程等待目标线程执行完毕。

3. 调用 `LockSupport.park()`。

• **进入 `TIMED_WAITING` (限时等待) 状态:**

1. 调用 `Thread.sleep(long millis)`, 线程“休眠”指定时间, 但不释放锁。

2. 调用 `Object.wait(long timeout)`, 在指定时间内等待, 超时或被唤醒后返回。

3. 调用 `Thread.join(long millis)`, 限时等待目标线程。

4. 调用 `LockSupport.parkNanos()` 或 `parkUntil()`。

1.2 线程控制方法

1. `wait()`, `sleep()`, `yield()` 和 `join()` 方法有什么区别? ★★★★★☆

方法	<code>wait()</code>	<code>sleep()</code>	<code>yield()</code>	<code>join()</code>
所属类	<code>Object</code>	<code>Thread</code>	<code>Thread</code>	<code>Thread</code>
是否释放锁	释放	不释放	不涉及锁	不涉及锁
使用前提	必须在 <code>synchronized</code> 代码块或方法中调用	任何地方	任何地方	任何地方
作用	使当前线程进入 WAITING 状态, 等待被唤醒。	使当前线程进入 TIMED_WAITING 状态, 暂停执行。	暂停 当前线程, 让出 CPU 给 同优先级 或更高优先级的线程, 自己回到 RUNNABLE (就绪) 状态。	使当前线程进入 WAITING 状态, 等待 目标线程执行完毕。
唤醒方式	被其他线程调用 <code>notify()</code> / <code>notifyAll()</code>	时间到期自动唤醒	CPU 重新调度	目标线程执行完毕自动唤醒

2. `wait()` 和 `notify()` / `notifyAll()` 方法为什么定义在 `Object` 类中? ★★★★★★

这是一个非常深刻的设计问题, 核心原因在于: **Java 中的锁是基于对象的, 而不是基于线程的。**

- 与锁的归属绑定:** `wait()` 和 `notify()` 是 Java 线程间通信的基石, 它们的本质是 **协调对共享资源的访问**。这种协调必须依赖一个“媒介”, 在 Java 中这个媒介就是 **对象锁 (也称为监视器, Monitor)**。
- 操作的是锁:** 线程调用 `wait()` 意味着“我因为某个条件不满足, 暂时放弃这个 **对象锁** 的持有权并进入等待队列”。调用 `notify()` 意味着“我改变了条件, 现在通知等待这个 **对象锁** 的其他线程来竞争”。

3. **Object 是锁的载体**: 既然这些操作都是围绕“某个对象上的锁”来进行的, 而Java中任何对象都可以作为锁, 那么将这些方法定义在所有对象的基类 `Object` 中, 就是最自然、最合理的设计。它保证了**任何对象**都能拥有充当锁和实现线程间通信的能力。

结论: `wait/notify` 不是操作线程, 而是操作持有该对象锁的线程。它们是**锁级别**的操作, 因此必须定义在所有锁的载体——`Object` 类中。

3. `Thread.sleep(0)` 的作用是什么? ★★★☆☆

`Thread.sleep(0)` 的作用在理论上是触发一次**操作系统级别的线程重新调度**。

调用它意味着当前线程想“暂停一下”, 主动放弃CPU时间片, 让操作系统的调度器有机会去执行其他处于就绪状态的线程。如果当前没有其他更高优先级的线程需要运行, 那么当前线程可能会被立刻重新调度回来。

在实践中, 它有时被用作一种“**hack**”手段, 来解决一些特定平台或场景下的线程饥饿问题, 或者在一些需要高精度计时的循环中, 插入 `sleep(0)` 来让出CPU, 避免空转。但它不是一个标准的、可移植的解决方案, 其具体行为高度依赖于操作系统和JVM的实现。

4. 如何优雅地终止一个线程? 为什么不推荐使用 `Thread.stop()`? ★★★★★

- **优雅终止的方式**: 使用**中断 (Interrupt)** 机制。

1. 在外部调用目标线程的 `thread.interrupt()` 方法。这只是一个协作请求, 它并不会直接停止线程, 而是设置线程的中断状态标志位为 `true`。
2. 在线程的 `run()` 方法内部, 通过循环检查 `Thread.currentThread().isInterrupted()` 的值。
3. 一旦发现中断标志为 `true`, 就安全地退出循环, 释放资源, 完成线程的终止。
4. 如果线程在 `wait()`, `sleep()`, `join()` 等方法中被阻塞, `interrupt()` 会使其抛出 `InterruptedException`, 可以在 `catch` 块中处理终止逻辑。

- 为什么不推荐 `Thread.stop()`:

`stop()` 方法已被废弃, 因为它极其不安全。它会像一颗“子弹”一样, 立即终止线程的执行, 并强行释放该线程持有的所有锁。

这可能导致数据处于不一致的状态。例如, 一个线程正在执行一个转账操作, 刚完成扣款, 还没来得及存款, 就被 `stop()` 终止, 它持有的锁被释放, 其他线程看到的就是一个数据被破坏的对象, 这会引发严重的逻辑错误。这种破坏是不可预测且难以修复的。

5. Java是如何判断一个线程是否存活的? ★★★☆☆

通过调用线程实例的 `isAlive()` 方法。

- 如果线程已经启动 (调用了 `start()`) 并且尚未终止 (`run()` 方法未执行完), 则 `isAlive()` 返回 `true`。
- 对于处于 `NEW` 状态和 `TERMINATED` 状态的线程, `isAlive()` 返回 `false`。

2. 线程创建与管理

2.1 线程创建方式

1. Java中创建线程有哪几种方式? 它们有什么区别和最佳实践? ★★★★★

主要有以下四种方式：

- 1. **继承 Thread 类:**
 - 创建一个类继承 Thread，并重写 run() 方法。
- 2. **实现 Runnable 接口:**
 - 创建一个类实现 Runnable 接口，并实现 run() 方法。将其实例作为参数传递给 new Thread()。
- 3. **实现 Callable 接口 (配合 FutureTask):**
 - 创建一个类实现 Callable 接口，并实现 call() 方法。call() 可以有返回值，也可以抛出异常。
 - 将 Callable 实例包装在 FutureTask 中，再将 FutureTask 作为参数传递给 new Thread()。
- 4. **使用线程池 (ExecutorService):**
 - 通过 Executors 工厂类创建线程池，然后调用 submit() 或 execute() 方法提交 Runnable 或 Callable 任务。

区别与最佳实践:

方式	优点	缺点	最佳实践
继承 Thread	简单直观	Java单继承限制，耦合度高	不推荐。
实现 Runnable	解耦（任务与线程分离），灵活（可继承其他类）	无返回值，不能抛出受检异常	推荐。符合面向接口编程，任务可以被多个线程复用。
实现 Callable	有返回值，可抛出异常	编码稍复杂	推荐。适用于需要异步计算结果的场景。
使用线程池	资源复用，性能高，统一管理	理解和配置稍复杂	强烈推荐。是生产环境中管理线程的标准实践，避免了频繁创建和销毁线程的开销。

2. 直接调用线程的 run() 方法与调用 start() 方法有什么区别？★★★★☆

这是一个核心区别，决定了是否开启新线程：

- **调用 start() 方法:**
 - 这才是启动一个新线程的正确方式。
 - 它会使线程进入就绪 (Ready) 状态，并由JVM的线程调度器在未来的某个时刻调用该线程的 run() 方法。
 - 这是一个异步调用。
- **直接调用 run() 方法:**
 - 这只是一个普通的成员方法调用。
 - 它不会创建新线程，而是在当前线程中同步地执行 run() 方法里的代码。

- 这是一个**同步**调用。

一句话总结: `start()` 是开启新线程并让JVM来调用 `run()` ; `run()` 只是在当前线程执行一个普通方法。

3. `Runnable` 和 `Callable` 接口有什么区别? `Future` 在其中扮演什么角色? ★★★★★

特性	<code>Runnable</code>	<code>Callable</code>
核心方法	<code>void run()</code>	<code>V call() throws Exception</code>
返回值	没有	有, 类型为泛型 <code>V</code>
异常处理	<code>run()</code> 不能抛出受检异常, 只能 <code>try-catch</code>	<code>call()</code> 可以向上抛出受检异常
使用方式	通常通过 <code>Thread</code> 或 <code>ExecutorService.execute()</code>	通常通过 <code>ExecutorService.submit()</code>

`Future` 的角色:

`Future` 是一个接口, 它代表一个异步计算的结果。

当你将一个 `Callable` 任务提交给线程池时, `submit()` 方法会立即返回一个 `Future` 对象。这个 `Future` 就像一张“提货单”, 你可以:

1. **查询状态:** 使用 `isDone()` 检查任务是否已完成。
2. **取消任务:** 使用 `cancel()` 尝试取消任务。
3. **获取结果:** 调用 `get()` 方法。如果任务已经完成, 它会立即返回结果; 如果任务尚未完成, `get()` 方法会**阻塞**当前线程, 直到任务完成并返回结果。

`Future` 巧妙地将**任务的提交**和**结果的获取**解耦开来, 使得主线程可以在等待结果的同时去处理其他事情, 是实现异步编程的关键。

2.2 线程类型与调度

1. 进程和线程的区别是什么? 多线程的优势体现在哪些方面? ★★★★★

• 进程 vs. 线程:

- **进程:** 是操作系统进行**资源分配和调度的基本单位**。它是一个独立的内存空间, 一个进程可以包含多个线程。
- **线程:** 是**CPU调度和执行的基本单位**。它也被称为轻量级进程, 是进程中的一个执行流。同一进程中的线程**共享**该进程的内存空间和资源。

比喻: 进程好比一个“工厂”, 拥有独立的厂房、设备和资源。线程则是工厂里的“工人”, 多个工人共享工厂的资源, 协同完成生产任务。

• 多线程的优势:

1. **提高CPU利用率:** 在等待I/O操作 (如读写文件、网络请求) 时, CPU可以切换到其他线程去执行计算任务, 避免空闲。
2. **提升程序响应速度:** 在GUI应用中, 可以将耗时的任务放在后台线程执行, 保持界面的流畅响应。

3. **并发处理任务**: 可以将一个大任务分解成多个小任务, 交由多个线程并行处理, 在多核CPU上能显著缩短总执行时间。

2. 什么是用户线程和守护线程? 它们对JVM的生命周期有什么影响? ★★★★★☆

- **用户线程 (User Thread)**: 我们通常创建的线程都是用户线程, 例如 `main` 线程。
- **守护线程 (Daemon Thread)**: 是一种特殊的后台线程, 它的主要任务是为其他用户线程提供服务。典型的例子是**垃圾回收 (GC) 线程**。

对JVM生命周期的影响:

这是它们最核心的区别: JVM会等待所有的用户线程执行完毕后会退出。

- 只要还有一个用户线程在运行, JVM就不会关闭。
- 如果只剩下守护线程在运行, JVM会**直接退出**, 而不会等待守护线程执行完毕。守护线程会随着JVM的关闭而立即终止。

可以通过 `thread.setDaemon(true)` 将一个线程设置为守护线程, 但这必须在调用 `start()` 方法之前完成。

3. 什么是线程调度? 上下文切换 (Context Switching) 会带来什么开销? ★★★★★☆

- **线程调度**: 指操作系统按照某种算法 (如时间片轮转、优先级调度等) 来为多个处于就绪状态的线程分配CPU时间片的过程。
- **上下文切换**: 当一个线程的时间片用完, 或者因等待I/O、锁等原因需要让出CPU时, 操作系统会**保存**当前线程的运行状态 (如程序计数器、寄存器值等), 这个状态被称为“上下文”。然后, 操作系统会**加载**下一个要执行的线程的上下文, 让其开始运行。这个保存和加载的过程就是**上下文切换**。
- **上下文切换的开销**:
 1. **直接开销**: CPU需要花费时间来执行保存和加载上下文的指令。
 2. **间接开销**: 上下文切换后, 新线程的代码和数据可能不在CPU缓存中, 导致缓存未命中 (Cache Miss) 增多, 需要从更慢的主存中加载数据, 这会降低程序的执行速度。

频繁的上下文切换是多线程程序性能下降的一个主要原因。

4. 什么是多线程中的上下文切换? ★★★★★☆

多线程中的上下文切换, 是指CPU从一个正在执行的线程, 转而去执行另一个处于就绪状态的线程的过程。在这个切换过程中, 操作系统必须:

1. **保存当前线程的状态**: 将当前线程的所有CPU寄存器中的值、程序计数器的位置等信息保存到内存中 (通常是该线程的内核栈)。
2. **加载下一个线程的状态**: 从内存中读取下一个即将运行的线程的上下文信息, 并将其加载到CPU的寄存器中。
3. **跳转执行**: 将CPU的指令指针跳转到新线程上次被中断的位置, 开始执行。

这个过程使得每个线程都感觉自己好像在独立地、连续地使用CPU, 但实际上它们是在CPU上快速地交替运行。

2.3 线程间数据共享与通信

1. 父子线程之间怎么共享/传递数据? ★★★☆☆

在Java中，父线程（创建者）和子线程（被创建者）之间共享数据主要有以下几种方式：

1. **通过构造函数或Setter方法传递**: 这是最直接、最简单的方式。在创建子线程（或其 `Runnable` 任务）时，将需要共享的数据作为参数传递给它构造函数。

```
// 父线程
String dataToShare = "Hello from parent";
MyRunnable task = new MyRunnable(dataToShare);
new Thread(task).start();
```

2. **使用 `final` 变量 (针对Lambda或匿名内部类)**: 如果在Lambda表达式或匿名内部类中创建线程，可以直接访问外部作用域中被 `final` 或事实最终 (effectively final) 修饰的局部变量。

```
final String sharedData = "Data";
new Thread(() -> {
    System.out.println("Child thread sees: " + sharedData);
}).start();
```

3. **通过共享的线程安全对象**: 定义一个共享对象（通常是线程安全的，如 `ConcurrentHashMap`, `BlockingQueue`, 或使用 `AtomicReference` 包装的对象），父子线程都持有该对象的引用，通过操作该对象来共享数据。这是实现复杂通信和数据交换的常用方式。
4. **使用 `InheritableThreadLocal`**: `ThreadLocal` 用于在单个线程内共享数据。其子类 `InheritableThreadLocal` 则可以在创建子线程时，将父线程中的值**自动复制**一份给子线程。注意，这只是创建时的一次性复制，之后父子线程各自的副本互不影响。

3. 线程同步机制 (synchronized, volatile, ReentrantLock)

3.1 synchronized 关键字

1. synchronized关键字的底层原理是什么? 它能修饰哪些类型? ★★★☆☆

- **底层原理**: `synchronized` 是JVM层面实现的，其核心依赖于每个对象都关联的一个**监视器 (Monitor)**。
 - **修饰代码块**: 通过 `monitorenter` 和 `monitorexit` 这两个字节码指令实现。当线程执行到 `monitorenter` 时，会尝试获取对象锁；`monitorexit` 则释放锁。
 - **修饰方法**: 通过在方法的元数据中添加 `ACC_SYNCHRONIZED` 访问标志来实现。当方法被调用时，JVM会检查此标志，如果是，则线程将先获取该方法所属对象的锁。
- **修饰范围**:
 1. **实例方法**: 锁住的是当前**实例对象 (this)**。
 2. **静态方法**: 锁住的是当前类的 `Class` 对象。
 3. **代码块**: 可以显式指定任何对象作为锁。

2. synchronized作用于实例方法和静态方法时，锁住的是什么？★★★★☆

- **作用于实例方法**：锁住的是**当前类的实例对象（this）**。不同实例对象之间的锁互不影响。
- **作用于静态方法**：锁住的是**当前类的 Class 对象**。因为一个类的 Class 对象是唯一的，所以无论有多少个实例，静态同步方法在同一时间也只能被一个线程访问。

3. synchronized是如何保证原子性、可见性、有序性的？★★★★★

- **原子性 (Atomicity)**：synchronized 保证其代码块内的操作是原子的、不可分割的。从 `monitorenter` 到 `monitorexit` 之间的所有指令，在执行期间不会被其他线程打断。
- **可见性 (Visibility)**：JMM (Java内存模型) 规定，线程在**释放锁**时，必须将自己工作内存中的所有共享变量值**刷新回主内存**。而在**获取锁**时，必须**清空工作内存**，并从主内存中重新加载共享变量的最新值。这一“解锁-刷新，加锁-加载”的机制，保证了线程间的可见性。
- **有序性 (Orderliness)**：synchronized 内部的代码允许发生指令重排序，但JMM保证线程在执行同步代码块时，会获得一种“仿佛是串行执行”的效果 (as-if-serial)。更重要的是，它隐式地禁止了**跨越同步块边界**的指令重排序，确保了程序的执行顺序在宏观上是正确的。

4. synchronized的锁升级过程是怎样的？★★★★☆

在JDK 1.6之后，synchronized 引入了锁升级机制以优化性能，过程是单向的，只能升级不能降级。

1. **偏向锁 (Biased Locking)**：当一个锁大多数时间都**只被同一个线程**获取时。在对象头中记录下该线程ID，后续该线程进入时无需加锁。
2. **轻量级锁 (Lightweight Locking)**：当出现**另一个线程**竞争时，偏向锁升级为轻量级锁。线程会通过**自旋 (Spinning)** 和**CAS**的方式尝试获取锁，避免立即阻塞。
3. **重量级锁 (Heavyweight Locking)**：如果自旋失败（竞争激烈），轻量级锁就会升级为重量级锁。依赖操作系统的**互斥量 (Mutex)**，获取不到锁的线程会被**阻塞**。

5. synchronized的重量级锁很慢，为什么还需要重量级锁？★★★☆☆

因为**轻量级锁的自旋是消耗CPU资源的**。

- 如果锁被持有的时间很短，自旋是值得的，因为它避免了上下文切换的昂贵开销。
- 但如果锁被持有的时间很长，让其他线程一直自旋空转，就会白白浪费CPU。
- 因此，**重量级锁**通过将等待线程**阻塞**，让出CPU给其他需要计算的线程，从而在**锁竞争激烈或锁持有时间长的**场景下，能获得更好的整体性能。

6. synchronized的锁优化是怎样的？★★★☆☆

除了锁升级，JVM在运行时还会进行其他优化：

1. **锁消除 (Lock Elision)**：JIT编译器如果检测到某个锁对象**不可能被共享**（如方法内的局部对象），就会**消除**这个不必要的锁。
2. **锁粗化 (Lock Coarsening)**：如果发现在一个循环中反复对同一个对象加锁和解锁，JIT编译器会**扩大锁的范围**，以减少加锁解锁的次数。
3. **自适应自旋 (Adaptive Spinning)**：在轻量级锁中，自旋的次数不是固定的。如果一个线程自旋成功过，JVM会认为下次成功的可能性也很大，并允许它自旋更长时间。

7. synchronized是非公平锁吗，那么是如何体现的？★★★★☆

是的，`synchronized` 是非公平锁。

- **体现**：当一个线程释放锁时，等待队列中的线程和新来的线程会**同时竞争**这把锁。JVM可能会将锁授予一个刚刚到达的线程，而不是已经在队列中等待了很久的线程。
- **原因**：这种“插队”行为虽然不公平，但能**减少CPU上下文切换**的次数，提高整体的**吞吐量**。

8. synchronized的锁能降级吗？★★★★☆

不能。`synchronized` 的锁升级过程（偏向锁 → 轻量级锁 → 重量级锁）是**单向的**，无法降级。这是为了简化锁机制的实现逻辑。

3.2 volatile 关键字

1. volatile关键字的作用是什么？它能保证原子性吗？为什么？★★★★☆

- **作用**：
 1. **保证可见性**：确保对 `volatile` 变量的修改能立即被其他线程看到。
 2. **禁止指令重排序**。
- **原子性**：`volatile` **不能保证复合操作的原子性**。
 - **为什么**：像 `count++` 这样的操作，包含“读-改-写”三个步骤。`volatile` 只能保证**单次的读或写**是原子的，但无法保证这三个步骤作为一个整体不被打断。

2. volatile如何保证可见性和禁止指令重排序？底层实现原理是什么？★★★★★

- **底层原理**：依赖于 **内存屏障（Memory Barriers）**。
 - **保证可见性**：在对 `volatile` 变量进行**写操作**时，JVM会在写指令后插入一个“**Store屏障**”，强制将工作内存的值刷新到主内存。在**读操作**时，会插入一个“**Load屏障**”，强制从主内存重新加载。
 - **禁止指令重排序**：内存屏障可以阻止其前后的指令越过屏障，从而保证了有序性。

3. int a = 1是原子性操作吗？★★★☆☆

是的。在标准的Java虚拟机中，对32位以内的数据类型（`boolean`, `byte`, `char`, `short`, `int`, `float`）以及引用类型的赋值操作，都被认为是**原子性**的。对于 `long` 和 `double`，使用 `volatile` 可以确保其读写操作的原子性。

4. 有了synchronized为什么还需要volatile？★★★★☆

因为 `volatile` 是一种**更轻量级**的同步选择，开销比 `synchronized` 小得多。

- **使用场景不同**：
 - 当一个变量的写入不依赖其当前值（如状态标志位），使用 `volatile` 就足以保证可见性，且性能更高。
 - `synchronized` 用于需要**互斥**访问的场景，即保护一个代码块，确保其中的一系列操作（如 `count++`）是原子的。
- **总结**：`volatile` 是“变量级别”的同步，`synchronized` 是“代码块级别”的同步。

5. 有了MESI为啥还要JMM? ★★★☆☆

- **MESI协议**: 是**硬件层面**的一种CPU缓存一致性协议, 保证多核CPU的缓存与主内存之间的数据一致。
- **JMM (Java内存模型)**: 是**语言规范层面**的抽象模型, 目的是**屏蔽底层硬件和编译器的差异**, 为Java开发者提供一个统一、跨平台的内存视图和并发编程规则。
- **关系**: JMM不仅要处理CPU缓存的问题, 还要处理**编译器的指令重排序**问题。JMM通过定义 `volatile` 等关键字的语义, 让JVM生成能在各种硬件平台上正确运行的指令 (包括插入必要的内存屏障, 而内存屏障的底层才会利用到MESI等硬件机制)。

3.3 synchronized 与 ReentrantLock 对比

1. synchronized和ReentrantLock有什么区别? 各自的适用场景是什么? ★★★★★

特性	synchronized	ReentrantLock
类型	Java关键字, JVM层面实现	API类 (<code>Lock</code>)
锁的释放	自动释放 (代码块结束或异常)	手动释放 (必须在 <code>finally</code> 块中调用 <code>unlock()</code>)
公平性	非公平锁	可选 , 可设置为公平锁
高级功能	无	支持 中断等待 、 超时等待 、 多条件变量

- **适用场景**:
 - `synchronized`: 适用于**简单、常规**的同步场景, 代码简洁, 不易出错, 是绝大多数情况下的首选。
 - `ReentrantLock`: 适用于需要**高级功能**的复杂场景。

2. ReentrantLock相比synchronized有哪些高级特性? ★★★★★

1. **可中断的锁获取** (`lockInterruptibly()`): 等待获取锁的过程中, 可以响应中断。
2. **可超时的锁获取** (`tryLock(long timeout, TimeUnit unit)`): 在指定时间内尝试获取锁, 超时则放弃。
3. **公平锁/非公平锁选择**: 可以在构造函数中指定锁是否是公平的。
4. **多条件变量** (`newCondition()`): 一个锁可以关联多个 `Condition` 对象, 实现对不同条件的精确唤醒。

3.4 并发编程基本特性

1. Java并发编程的三大特性 (原子性、可见性、有序性) 分别指什么? 如何保证? ★★★★★

- **原子性 (Atomicity)**: 指一个或多个操作, 要么全部执行成功, 要么全部不执行, 中间不能被任何因素打断。
 - **保证方式**: `synchronized`、`Lock`、`java.util.concurrent.atomic` 包下的原子类。
- **可见性 (Visibility)**: 指当一个线程修改了共享变量的值, 其他线程能够立即得知这个修改。
 - **保证方式**: `volatile`、`synchronized`、`Lock`、`final` 关键字。
- **有序性 (Orderliness)**: 指程序执行的顺序按照代码的先后顺序执行。

- **保证方式:** `volatile` (禁止重排序)、`synchronized` 和 `Lock` (通过互斥间接保证)。

2. 并发编程中的原子性和数据库ACID的原子性一样吗? ★★★☆☆

不一样。它们概念上相似,但**范围和目的**完全不同。

- **并发编程的原子性:** 关注的是**单个或多个操作**相对于**其他线程**的不可分割性,解决的是**线程调度**层面的问题。
- **数据库ACID的原子性:** 关注的是一个**事务 (Transaction)**,保证整个事务要么全部成功提交,要么在失败时**全部回滚**,解决的是**数据持久化和完整性**的问题。

3. 有哪些实现线程安全的方案? ★★★★★☆

主要有三大类方案:

1. 互斥同步 (悲观锁):

- **核心理念:** 假定总会发生冲突,访问前先加锁。
- **实现:** `synchronized`、`ReentrantLock`。

2. 非阻塞同步 (乐观锁):

- **核心理念:** 假定不会发生冲突,直接尝试修改,若冲突则重试。
- **实现:** 基于**CAS (Compare-And-Swap)** 原子指令,如 `java.util.concurrent.atomic` 包下的类。

3. 无同步方案:

- **核心理念:** 从根本上避免共享数据。
- **实现:**
 - **不可变对象 (Immutability):** 使用 `final` 关键字创建不可变对象。
 - **线程本地存储 (ThreadLocal):** 为每个线程提供一个独立的变量副本。

4. Java内存模型 (JMM) 与happens-before原则

4.1 Java内存模型 (JMM)

1. 什么是Java内存模型 (JMM)? 它解决了什么问题? ★★★☆☆

- **是什么:** Java内存模型 (JMM) 是一种**规范**,它定义了Java程序中各种变量(线程共享变量)的访问规则,以及在多线程环境下,这些变量在内存中的读写操作所应遵循的底层细节。
- **解决了什么问题:** JMM主要解决了由CPU缓存、编译器优化和指令重排序等因素导致的**可见性和有序性问题**。它通过定义一套规则(如 `happens-before`),屏蔽了底层硬件和编译器的差异,为Java开发者提供了一个统一、跨平台的并发编程内存视图。

2. JMM中主内存和工作内存的概念是什么? 线程对共享变量的操作是如何进行的? ★★★★★☆

JMM定义了一种抽象的内存结构,包含主内存和工作内存:

- **主内存 (Main Memory):** 是所有线程**共享**的区域,存储了所有的实例字段、静态字段等共享变量。
- **工作内存 (Working Memory):** 是每个线程**私有**的区域。线程对变量的所有操作(读取、赋值等)都必须在工作内存中进行。

操作流程:

1. **读取 (Load & Read):** 线程首先从主内存中将共享变量的值**加载**到自己的工作内存中。

2. **使用 (Use)** : 线程在工作内存中使用这个变量副本。

3. **修改 (Assign & Store & Write)** : 线程修改变量后, 会将新值**存储**在工作内存中, 并在未来的某个时刻**写回**到主内存。

线程之间无法直接访问对方的工作内存, 所有通信都必须通过主内存来完成。这种模型解释了为什么在没有同步措施的情况下, 一个线程对共享变量的修改对另一个线程是不可见的。

3. 什么是总线嗅探和总线风暴, 和JMM有什么关系? ★★★★★☆

这是与CPU缓存一致性协议 (如MESI) 相关的硬件层概念, JMM的可见性保证间接依赖于这些机制。

- **总线嗅探 (Bus Snooping)** : 是实现缓存一致性的一种机制。每个CPU核心都会持续“嗅探”总线上发生的数据交换, 并检查这些数据是否与自己缓存中的数据相关。如果一个核心发现有其他核心修改了某个共享数据 (该数据也在自己的缓存中), 它就会将自己缓存中对应的缓存行标记为**无效**。下次访问该数据时, 就必须从主内存重新加载。这是 `volatile` **实现可见性的硬件基础**。
- **总线风暴 (Bus Storm)** : 当大量CPU核心同时、反复地修改共享数据时, 会导致大量的缓存失效和总线通信, 使得总线不堪重负, 严重影响系统性能。这就是总线风暴。
- **与JMM的关系**: JMM通过定义 `volatile` 和 `synchronized` 等关键字的内存语义, 来指导JVM生成能够正确利用底层硬件 (如总线嗅探机制) 的指令, 以确保可见性。同时, JMM的设计也在试图避免过度依赖可能导致总线风暴的底层操作, 例如, `synchronized` 通过互斥访问减少了并发写的频率。

4.2 happens-before原则

1. 什么是happens-before原则? 它在JMM中有什么作用? ★★★★★☆

- **是什么**: `happens-before` 原则是JMM中用于**判断数据是否存在竞争、线程是否安全**的核心依据。它定义了两个操作之间的**偏序关系**。如果操作A `happens-before` 操作B, 那么JMM就向程序员保证: A操作的结果对B操作是可见的, 并且A操作的执行顺序在B操作之前。
- **作用**: 它极大地简化了并发编程。开发者无需关心底层复杂的重排序和内存可见性问题, 只需根据 `happens-before` 规则来推断程序的执行顺序和可见性, 就可以编写出正确的并发程序。

2. 列举并解释几种常见的happens-before规则。★★★★★

1. **程序次序规则 (Program Order Rule)** : 在一个线程内, 按照代码的先后顺序, 书写在前面的操作 `happens-before` 书写在后面的操作。
2. **管程锁定规则 (Monitor Lock Rule)** : 一个 `unlock` 操作 `happens-before` 后续对**同一个锁**的 `lock` 操作。
3. **volatile变量规则 (Volatile Variable Rule)** : 对一个 `volatile` 变量的**写**操作 `happens-before` 后续对这个变量的**读**操作。
4. **线程启动规则 (Thread Start Rule)** : `Thread` 对象的 `start()` 方法 `happens-before` 此线程的任何一个动作。
5. **线程终止规则 (Thread Termination Rule)** : 线程中的所有操作都 `happens-before` 对此线程的终止检测 (如 `thread.join()` 或 `thread.isAlive()` 返回 `false`) 。
6. **传递性 (Transitivity)** : 如果操作A `happens-before` 操作B, 操作B `happens-before` 操作C, 那么可以得出操作A `happens-before` 操作C。

3. volatile和synchronized如何根据happens-before原则保证可见性和有序性? ★★★★★

- **volatile:**
 - **可见性与有序性:** 依赖**volatile变量规则**。当线程A写入一个volatile变量，线程B读取该变量时，根据规则，A的写操作 happens-before B的读操作。这意味着A在写volatile变量之前的所有操作结果，对B在读volatile变量之后都是可见的。
- **synchronized:**
 - **可见性与有序性:** 依赖**管程锁定规则**。当线程A释放锁，线程B获取同一个锁时，A的解锁操作 happens-before B的加锁操作。这意味着A在解锁之前的所有操作结果，对B在加锁之后都是可见的。

4. happens-before和as-if-serial有啥区别和联系? ★★★★★☆

- **as-if-serial:** 意为“仿佛是串行的”。它是一个**单线程**内的规则，保证在单线程程序中，无论编译器和处理器如何进行重排序，最终的执行结果都必须和代码顺序执行的结果一致。它给了编译器和处理器极大的优化空间。
- **happens-before:** 是**多线程**间的规则，它约束了编译器的优化行为，保证了操作在多线程之间的可见性和有序性。
- **区别与联系:**
 - as-if-serial 是 happens-before 的基础。happens-before 关系中的两个操作如果处于同一个线程，那么它们也必须遵循 as-if-serial。
 - as-if-serial 保证单线程程序的正确性，而 happens-before 保证正确同步的多线程程序的正确性。

5. JUC并发包 (java.util.concurrent)

5.1 JUC 核心组件

1. JUC并发包中包含哪些核心组件? 相比传统并发机制有什么优势? ★★★☆☆

- **核心组件:**
 1. **锁 (Locks)** : 提供了比 synchronized 更灵活的锁机制，如 ReentrantLock, ReadWriteLock。
 2. **原子类 (Atomic)** : 提供了基于CAS的原子操作类，如 AtomicInteger, AtomicLong。
 3. **并发集合 (Concurrent Collections)** : 提供了高性能的线程安全集合，如 ConcurrentHashMap, CopyOnWriteArrayList。
 4. **并发工具类 (Synchronizers)** : 提供了用于线程协作的工具，如 CountdownLatch, CyclicBarrier, Semaphore。
 5. **执行器与线程池 (Executors & ThreadPool)** : 提供了强大的线程池框架，用于管理和调度线程。
- **优势:** 相比 synchronized / wait / notify 等传统机制，JUC提供了**性能更高、功能更强大、使用更灵活**的并发解决方案，让开发者能够更容易地编写出健壮、高开发的程序。

5.2 并发工具类

1. Semaphore（信号量）的用途是什么？它是如何实现资源访问控制的？★★★★☆

- **用途:** Semaphore 用于控制**同时访问某个特定资源的线程数量**。它常用于实现资源池（如数据库连接池）或流量控制。
- **实现原理:** Semaphore 内部维护一个**许可证 (permit)** 计数器。
 - `acquire()`: 线程调用该方法尝试获取一个许可证。如果计数器大于0，则获取成功，计数器减1；如果计数器为0，则线程被阻塞，直到有其他线程释放许可证。
 - `release()`: 线程调用该方法释放一个许可证，计数器加1，并可能唤醒一个正在等待的线程。

2. CountdownLatch、CyclicBarrier、Semaphore 区别？★★★★☆

这三者是JUC中最重要并发协作工具，区别非常明显。

特性	CountDownLatch	CyclicBarrier	Semaphore
核心功能	一个或多个线程 等待 其他一组线程完成操作	一组线程 相互等待 ，直到所有线程都到达一个公共的屏障点	控制同时访问特定资源的 线程数量
计数器	<code>countDown()</code> 使计数器减1	<code>await()</code> 使计数器减1（内部）	<code>release()</code> 使许可证加1
阻塞点	<code>await()</code> 方法阻塞，直到计数器为0	<code>await()</code> 方法阻塞，直到所有线程都到达屏障	<code>acquire()</code> 方法阻塞，直到有可用许可证
可重用性	不可重用 ，计数器到0后无法重置	可重用 ，所有线程越过屏障后，计数器会自动重置	可看作可重用的，许可证可以被反复获取和释放
典型场景	主线程等待所有子任务完成	多个线程分阶段、同步执行任务	资源池、流量控制

3. CountdownLatch 和 CyclicBarrier 有什么区别？它们各自的典型应用场景是什么？★★★★☆

这是对上一个问题的细化，主要区别在于：

- **功能焦点不同:**
 - `CountDownLatch`: 强调的是**一个线程等待**其他多个线程完成事件。是“多等一”或“一等多”的模式。
 - `CyclicBarrier`: 强调的是**多个线程相互等待**，直到所有线程都准备就绪再一起继续执行。是“多等（等够数）多”的模式。
- **可重用性:**

- `CountDownLatch` 的计数器只能使用一次。
- `CyclicBarrier` 的计数器可以被重置（通过 `reset()` 或自动），因此可以循环使用。
- 场景:
 - `CountDownLatch`: 适用于一个任务需要等待其他依赖任务全部完成后才能开始的场景。例如，主线程需要等待所有初始化任务加载完毕。
 - `CyclicBarrier`: 适用于需要将一个大任务拆分成多个子任务，且这些子任务需要同步进行，按阶段推进的场景。例如，在大型计算中，所有线程计算完第一阶段的数据后，需要同步交换数据，然后再一起进入第二阶段。

5.3 并发集合

1. 什么是 `BlockingQueue`（阻塞队列）？它有哪些常见实现？虚假唤醒问题是什么？★★★★☆

- 是什么: `BlockingQueue` 是一个接口，它在标准队列的基础上增加了**阻塞**特性。
 - 当队列满时，入队操作会阻塞生产者。
 - 当队列空时，出队操作会阻塞消费者。
- 常见实现:
 - `ArrayBlockingQueue`: 基于数组的有界阻塞队列。
 - `LinkedBlockingQueue`: 基于链表的、通常是无界的阻塞队列。
 - `SynchronousQueue`: 不存储元素的阻塞队列，每个 `put` 操作必须等待一个 `take` 操作。
- 虚假唤醒 (**Spurious Wakeup**): 指线程在没有被 `notify/signal` 的情况下，从 `wait/await` 状态中被意外唤醒。这是一个罕见但可能发生的情况。为了防止因此导致的错误，官方推荐始终在 `while` 循环中检查等待条件，而不是在 `if` 语句中。

```
// 错误的方式
if (condition) {
    wait();
}
// 正确的方式
while (condition) {
    wait();
}
```

2. `ConcurrentHashMap` 的并发实现原理在JDK 1.7和JDK 1.8中有什么不同？★★★★★

- JDK 1.7:
 - 原理: **分段锁 (Segment Lock)**。 `ConcurrentHashMap` 内部由一个 `Segment` 数组组成，每个 `Segment` 都是一个独立的、继承自 `ReentrantLock` 的小 `HashMap`。
 - 并发控制: 对数据进行操作时，只需锁住其所在的 `Segment` 即可，不同 `Segment` 之间的操作可以并行，大大提高了并发度。
- JDK 1.8:
 - 原理: **放弃了分段锁**，回归到与 `HashMap` 类似的“数组 + 链表 / 红黑树”结构。
 - 并发控制: 锁的粒度细化到**每个桶的头节点**。写操作时，如果桶为空，使用 **CAS** 尝试添加；如果桶不为空，则使用 `synchronized` 锁住头节点进行操作。读操作完全不加锁，通过 `volatile` 保证可见性。
 - 优势: 锁的粒度更细，并发性能更高，且实现更简洁。

3. CopyOnWriteArrayList 的“写时复制”机制是什么？它的优缺点和适用场景？★★★★☆

- **写时复制机制:** CopyOnWriteArrayList (COW) 是一种读写分离的并发策略。
 - **读操作:** 完全不加锁，直接访问底层数组。
 - **写操作 (add, set, remove):**
 1. 先加锁，保证同一时间只有一个写操作。
 2. **复制**一份当前底层数组的副本。
 3. 在新副本上进行修改。
 4. 将内部指向数组的引用**原子地**切换到新副本上。
- **优缺点:**
 - **优点:** 读操作性能极高，因为完全无锁，非常适合读远多于写的场景。
 - **缺点:**
 - **内存占用大:** 每次写操作都会复制整个数组。
 - **数据一致性是最终一致性:** 写操作和读操作在不同数组上，读线程可能无法立即看到最新的修改。
- **适用场景:** 读多写少的并发场景。最典型的例子就是事件监听器 (Listener) 列表的管理。

6. 线程池 (ThreadPoolExecutor) 核心原理

6.1 线程池基础

1. 为什么需要线程池？使用线程池有哪些优势？★★★★☆

- **为什么需要:** 在程序中频繁地创建和销毁线程会带来巨大的开销。线程池通过复用已创建的线程来解决这个问题。
- **优势:**
 1. **降低资源消耗:** 通过复用线程，减少了线程创建和销毁的开销。
 2. **提高响应速度:** 任务到达时，无需等待线程创建即可立即执行。
 3. **提高线程的可管理性:** 线程是稀缺资源，如果无限制地创建，不仅会消耗系统资源，还会降低系统的稳定性。线程池可以进行统一的分配、调优和监控。

2. 什么是线程池？如何实现的？★★★★☆

- **是什么:** 线程池是一种管理和复用线程的机制。它内部维护一个线程集合 (“池”) 和一个任务队列。当需要执行异步任务时，将任务提交给线程池，线程池会从池中取出一个空闲线程来执行该任务。
- **如何实现:** Java中的线程池核心实现是 `java.util.concurrent.ThreadPoolExecutor` 类。它通过管理一组**工作线程 (Worker)**、一个**工作队列 (workQueue)** 和一套**拒绝策略 (RejectedExecutionHandler)** 来实现。工作线程会循环地从工作队列中获取任务来执行。

6.2 ThreadPoolExecutor 参数与工作流程

1. ThreadPoolExecutor的七个核心参数分别代表什么含义？它们如何影响线程池的行为？★★★★★

1. `corePoolSize` (**核心线程数**): 线程池中保持存活的最小线程数，即使它们是空闲的。
2. `maximumPoolSize` (**最大线程数**): 线程池允许创建的线程总数上限。

3. **keepAliveTime (存活时间)**: 当线程池中的线程数超过 `corePoolSize` 时, 多余的空闲线程在被销毁前可以存活的时间。
4. **unit (时间单位)**: `keepAliveTime` 的时间单位 (如秒、毫秒)。
5. **workQueue (工作队列)**: 用于暂存待执行任务的**阻塞队列**。其类型 (有界/无界) 会极大影响线程池行为。
6. **threadFactory (线程工厂)**: 用于创建新线程的工厂, 可以自定义线程名称、是否为守护线程等。
7. **handler (拒绝策略)**: 当线程池和工作队列都满了之后, 新任务的处理策略。

2. ThreadPoolExecutor的工作流程是怎样的? 任务是如何被提交和执行的? ★★★★★☆

1. 当一个新任务通过 `execute()` 提交时, 线程池首先判断当前运行的线程数是否**小于** `corePoolSize`?
 - 是 -> 创建一个新的**核心线程**来执行该任务。
 - 否 -> 进入下一步。
2. 判断**工作队列** `workQueue` **是否已满**?
 - 否 -> 将任务**加入工作队列**。
 - 是 -> 进入下一步。
3. 判断当前运行的线程数是否**小于** `maximumPoolSize`?
 - 是 -> 创建一个新的**非核心线程**来执行该任务。
 - 否 -> 执行**拒绝策略** `handler`。

3. 线程数设定成多少更合适? ★★★★★☆

这是一个需要根据任务类型来决定的经典问题:

- **CPU密集型任务** (如加密、计算): 这类任务消耗大量CPU资源。应将线程数设置得尽可能小, 通常为 `CPU核心数 + 1`。这样可以减少因线程切换带来的上下文开销。
- **I/O密集型任务** (如数据库读写、网络请求): 这类任务大部分时间在等待I/O操作完成, CPU处于空闲状态。可以设置更多的线程数, 让CPU在等待一个线程的I/O时, 去执行其他线程。一个常见的经验公式是 `CPU核心数 * 2` 或 `CPU核心数 / (1 - 阻塞系数)`。

4. 线程池如何实现线程复用? 请解释其底层原理。 ★★★★★☆

线程池实现复用的核心在于其内部的**工作线程 (worker 类)**。

- `Worker` 是 `ThreadPoolExecutor` 的一个内部类, 它实现了 `Runnable` 接口, 并且其自身也包含一个 `Thread` 对象。
- 当一个 `worker` 启动后, 它会进入一个**循环**。在循环中, 它会不断地尝试从**工作队列** `workQueue` 中通过 `take()` 或 `poll()` 方法获取任务。
- 获取到任务后, 就执行任务的 `run()` 方法。
- **任务执行完毕后**, `worker` **不会终止**, 而是继续下一次循环, 尝试获取下一个任务。
- 通过这种方式, 线程 (`worker` 中的 `Thread` 对象) 被持续复用, 直到线程池决定将其销毁 (例如, 当线程空闲时间超过 `keepAliveTime`)。

5. 如何合理设置线程池的核心参数，特别是针对CPU密集型和I/O密集型任务？★★★★★

• CPU密集型任务:

- `corePoolSize` & `maximumPoolSize`: 设置为 CPU核心数 + 1。
- `workQueue`: 通常使用**有界队列**，如 `ArrayBlockingQueue`，容量不宜过大。
- `keepAliveTime`: 可以设置得较长，因为核心线程通常不希望被销毁。

• I/O密集型任务:

- `corePoolSize` & `maximumPoolSize`: 设置为**较大的值**，如 CPU核心数 * 2 或根据业务压测结果调整。
- `workQueue`: 可以使用容量更大的有界队列，或者在某些场景下使用 `SynchronousQueue` 来快速创建非核心线程。
- `keepAliveTime`: 可以设置得短一些，以便在业务低谷期回收空闲线程。

- **通用建议**: 始终使用**有界队列**，并根据业务的QPS和任务平均耗时来估算队列容量和线程数，避免因无界队列导致内存溢出。

6. 为什么不建议通过Executors构建线程池？★★★☆☆

因为 `Executors` 提供的几个静态工厂方法（如 `newFixedThreadPool`，`newCachedThreadPool`）存在潜在的**资源耗尽风险**:

- `newFixedThreadPool` 和 `newSingleThreadExecutor`: 其内部使用了一个**无界的** `LinkedBlockingQueue`。如果任务生产速度远快于消费速度，会导致队列中积压大量任务，可能引发**内存溢出 (OOM)**。
- `newCachedThreadPool`: 其 `maximumPoolSize` 设置为 `Integer.MAX_VALUE`，允许创建几乎无限多的线程。在高并发下，可能会创建大量线程，同样可能导致**内存溢出**。
- **最佳实践**: 阿里巴巴《Java开发手册》中强制要求，必须通过 `ThreadPoolExecutor` 的构造函数来创建线程池，这样可以对所有参数有明确的控制，让开发者更清楚线程池的运行规则。

6.3 线程池状态与管理

1. 线程池有哪些核心状态？`shutdown()` 和 `shutdownNow()` 方法有什么区别？★★★★☆

线程池有5个状态:

1. **RUNNING**: 正常运行，能接受新任务，也能处理队列中的任务。
2. **SHUTDOWN**: 不再接受新任务，但会处理完队列中已有的任务。调用 `shutdown()` 后进入此状态。
3. **STOP**: 不接受新任务，不处理队列中的任务，并中断正在执行的任务。调用 `shutdownNow()` 后进入此状态。
4. **TIDYING**: 所有任务都已终止，工作线程数量为0，即将调用 `terminated()` 方法。
5. **TERMINATED**: `terminated()` 方法执行完毕。

• `shutdown()` 与 `shutdownNow()` 的区别:

- `shutdown()` (**优雅关闭**): 不再接受新任务，但会等待已提交的任务（包括队列中的）全部执行完成。
- `shutdownNow()` (**暴力关闭**): 立即停止所有正在执行的任务（通过发送中断信号），清空工作队列，并返回未执行的任务列表。

2. 线程池的四种拒绝策略分别是什么？如何自定义拒绝策略？★★★★☆

1. `AbortPolicy` (默认策略): 直接抛出 `RejectedExecutionException` 异常, 阻止系统正常运行。
 2. `CallerRunsPolicy`: 将任务回退给调用者线程来执行。如果主线程提交任务被拒绝, 则主线程自己会去执行这个任务, 这是一种降级策略。
 3. `DiscardPolicy`: 直接丢弃任务, 不做任何处理。
 4. `DiscardOldestPolicy`: 丢弃工作队列中等待时间最长的任务, 然后把当前任务加入队列。
- **如何自定义**: 实现 `RejectedExecutionHandler` 接口, 并重写 `rejectedExecution` 方法。可以在此方法中实现自定义的逻辑, 如记录日志、持久化任务、发送告警等。

3. 线程池是如何被调度的？★★★★☆

线程池的调度指的是**任务的调度**, 而非线程本身被操作系统调度。任务的调度由线程池的工作流程决定:

1. 优先由**核心线程**执行。
2. 核心线程满了, 任务进入**工作队列**排队。
3. 队列满了, 尝试创建**非核心线程**执行。
4. 都满了, 执行拒绝策略。

这个流程体现了线程池在“立即执行”和“排队等待”之间的权衡策略。

4. `execute()`和`submit()`方法有什么区别？在异常处理方面有何不同？★★★★☆

- `execute(Runnable command)`:
 - **参数与返回**: 只能接收 `Runnable` 任务, 没有返回值。
 - **异常处理**: 如果任务在执行过程中抛出异常, 会直接抛出到执行该任务的线程中, 通常会导致该线程终止。
- `submit(Runnable task)` 或 `submit(Callable<T> task)`:
 - **参数与返回**: 可以接收 `Runnable` 和 `Callable` 任务, 并返回一个 `Future` 对象。
 - **异常处理**: 任务执行中抛出的任何异常都会被**捕获**并存储在 `Future` 对象中。只有当调用 `Future.get()` 方法时, 这个异常才会被重新抛出。这使得异常处理更加可控。

5. `CompletableFuture`的底层是如何实现的？★★★★☆

`CompletableFuture` 是对 `Future` 的极大增强, 其核心是**回调机制**和**异步任务编排**。

- **底层实现**: 它的异步操作 (如 `supplyAsync`, `runAsync`) 默认依赖于公共的 `ForkJoinPool.commonPool()` 线程池来执行。
- **回调机制**: `CompletableFuture` 内部维护了一个类似于**链表**的结构, 来存储后续的依赖操作 (如 `thenApply`, `thenAccept`)。当一个阶段的计算完成后, 它会遍历这个链表, 触发并执行下一个阶段的任务。
- **状态管理**: 通过CAS操作来原子地更新其内部状态 (如 `result` 字段), 并通知后续依赖的任务。

6. `ForkJoinPool`和`ThreadPoolExecutor`区别是什么？★★★★☆

- `ThreadPoolExecutor`: 是一个**通用**的线程池, 适合执行各种独立的、耗时差不多的任务。
- `ForkJoinPool`: 是一个**专用**的线程池, 专为**分治 (Fork/Join)** 算法设计。

- **核心区别:** `ForkJoinPool` 实现了**工作窃取 (Work-Stealing)** 算法。当池中的某个线程完成了自己的所有任务后, 它会去“窃取”其他线程任务队列末尾的任务来执行, 以最大限度地利用 CPU。这使得它非常适合处理能够被递归分解成许多小任务的计算密集型问题。

7. 如何让Java的线程池顺序执行任务? ★★★☆☆

最简单的方式是使用一个**单线程的线程池**。

- 通过 `Executors.newSingleThreadExecutor()` 来创建一个线程池。
- 这个线程池的 `corePoolSize` 和 `maximumPoolSize` 都为1, 并使用一个无界的 `LinkedBlockingQueue`。
- 由于只有一个工作线程, 所有提交的任务都会进入队列中, 并由这个唯一的线程**按顺序**从队列中取出并执行, 从而保证了任务的顺序执行。

7. 原子类 (Atomic 包) 与CAS操作

7.1 原子类与CAS 基础

1. 什么是原子类 (Atomic Classes) ? 它们的作用是什么? ★★★☆☆

- **是什么:** `java.util.concurrent.atomic` 包下的一系列类, 它们提供了一种在多线程环境下进行**无锁 (lock-free)** 的、**原子性**的操作。
- **作用:** 用于在不使用 `synchronized` 或 `Lock` 的情况下, 实现对单个变量 (如计数器、状态标志) 的线程安全更新。常见的有 `AtomicInteger`, `AtomicLong`, `AtomicBoolean` 等。

2. 什么是CAS (Compare-And-Swap) 操作? 请解释其原理和三个核心参数。★★★★☆

- **是什么:** CAS (比较并交换) 是一条**CPU原子指令**, 是实现无锁并发的核心技术。
- **原理:** 它包含一个不可分割的操作序列:
 1. 从内存中读取一个值。
 2. 与一个期望值进行比较。
 3. 如果相等, 则将内存中的值更新为一个新值。
 4. 如果不相等, 则不做任何操作。
- **三个核心参数:**
 1. **V:** 内存地址 (要操作的变量) 。
 2. **A:** 期望的旧值 (Expected Value) 。
 3. **B:** 要更新的新值 (New Value) 。

操作语义: `if (V == A) { V = B; } else { // do nothing }`, 整个过程是原子的。

3. 什么是CAS? 存在什么问题? ★★★☆☆

- **是什么:** 见上题。
- **存在的问题:**
 1. **ABA问题:** 这是CAS最著名的问题。详见后文。
 2. **自旋开销:** 如果竞争激烈, CAS操作会反复失败, 线程会持续进行**自旋**重试, 这会消耗大量 CPU资源。
 3. **只能保证单个共享变量的原子操作:** 对于多个共享变量的操作, CAS无能为力, 仍然需要使用锁。

4. CAS是如何实现无锁 (Lock-Free) 并发的? 它有哪些优缺点? ★★★☆☆

- **如何实现:** CAS利用CPU提供的原子指令, 避免了使用操作系统层面的互斥锁。线程在修改数据时, 不是去获取锁, 而是乐观地假设没有冲突, 直接尝试用CAS指令去更新。如果成功, 则操作完成; 如果失败 (说明值被其他线程改了), 则通常会进入一个**循环 (自旋)**, 重新读取新值, 再次尝试CAS, 直到成功为止。
- **优缺点:**
 - **优点:**
 - **高性能:** 避免了线程阻塞和上下文切换的开销。
 - **无死锁:** 因为没有锁, 所以不会产生死锁问题。
 - **缺点:**
 - **自旋消耗CPU:** 竞争激烈时, 长时间自旋会浪费CPU。
 - **ABA问题。**

5. CAS在操作系统层面是如何保证原子性的? ★★★☆☆

CAS的原子性是由**硬件**直接保证的。现代CPU都提供了一条特殊的原子指令 (如x86的 `cmpxchg` 指令), 这条指令在执行期间可以**锁定总线**或使用其他缓存一致性机制, 来确保“比较”和“交换”这两个步骤作为一个不可分割的单元被执行, 不会被其他CPU核心的操作所干扰。

6. 有了CAS为啥还需要volatile? ★★★☆☆

因为它们解决的问题不同, 通常是**配合使用**的。

- `volatile` 保证的是**可见性和有序性**。它确保线程在进行CAS操作前, 能读取到共享变量的**最新值** (即期望值A) 。
- `CAS` 保证的是**原子性**。它确保“比较并更新”这个复合操作是不可分割的。
- 在 `AtomicInteger` 的 `getAndIncrement()` 实现中, 其内部的 `value` 字段就是被 `volatile` 修饰的, 以保证每次循环都能拿到最新的值来参与CAS。

7.2 CAS 常见问题

1. 什么是ABA问题? 如何解决ABA问题? ★★★★★

- **什么是ABA问题:** CAS在操作时只检查“期望值A”是否被改变, 但无法感知它是否被改成了其他值又被改回了A。
 - **例子:** 线程1准备将值从A更新为B。它读取到值A。此时, 线程2将值从A改为C, 然后又改回A。线程1回来执行CAS时, 发现内存值仍然是A, 于是成功更新为B。但实际上, 数据已经发生了变化, 这个CAS操作可能违背了业务逻辑的初衷。
- **如何解决:**
 - 通过引入**版本号**或**时间戳**。
 - JUC包提供了 `AtomicStampedReference` 类, 它在CAS时不仅会检查当前值, 还会检查一个**版本戳 (stamp)**。只有当值和版本戳都符合期望时, 才会执行更新, 并在更新后将版本戳加1。

2. 乐观锁和悲观锁的区别是什么？CAS属于哪种类型？ ★★★☆☆

- 悲观锁 (Pessimistic Locking) :

- 思想: 总是假设最坏的情况, 认为数据在每次被访问时都会被其他线程修改, 所以每次操作前都先加锁。
- 代表: `synchronized`, `ReentrantLock`。

- 乐观锁 (Optimistic Locking) :

- 思想: 总是假设最好的情况, 认为数据在被访问时不会被修改, 所以操作前不加锁。在更新时才去判断在此期间数据是否被其他线程修改过。
- 代表: CAS 就是乐观锁的一种典型实现。

- 结论: CAS属于乐观锁。

3. CAS一定有自旋吗？ ★★★☆☆

不一定。

- CAS本身只是一次性的原子操作, 它执行完就返回成功或失败, 并不包含循环。
- 我们常说的“CAS操作”通常指的是一个基于CAS实现的无锁算法, 例如 `AtomicInteger.getAndIncrement()`。在这种算法中, 为了保证操作最终能成功, 通常会把CAS操作放在一个 `do-while` 循环里, 这就形成了自旋。
- 但如果一个业务场景只允许尝试一次, 失败就立即返回, 那么就可以只调用一次CAS而不自旋。

4. 什么是Unsafe？ ★★★☆☆

`sun.misc.Unsafe` 是一个特殊的、隐藏在JDK深处的类。它提供了非常底层的、类似于C++指针操作的功能, 可以直接操作内存。

- 它绕过了JVM的安全检查, 因此使用它是不安全的, 不推荐在普通应用代码中使用。
- 在JUC中, `Unsafe` 类被广泛用于实现高性能的并发组件。例如, 所有原子类 (`AtomicInteger` 等) 底层的CAS操作, 就是通过调用 `Unsafe` 类提供的 `compareAndSwapInt` 等native方法来实现的。可以说, `Unsafe` 是Java实现CAS操作的基石。

8. 锁机制 (公平/非公平锁, 读写锁, AQS)

8.1 AQS 详解

1. 什么是AQS (AbstractQueuedSynchronizer) ? 它的核心组件 (state变量、CLH队列) 是什么？ ★★★★★

- 是什么: AQS (抽象队列同步器) 是JUC (`java.util.concurrent`) 包下所有锁和同步工具 (如 `ReentrantLock`, `Semaphore`) 的核心实现框架。它提供了一个通用的、基于队列的同步机制, 开发者可以通过继承它来轻松实现自定义的同步器。
- 核心组件:
 1. state 变量: 一个 `volatile int` 类型的变量, 用于表示同步状态。0 通常表示未锁定, 大于0表示已锁定。子类通过 `getState()`, `setState()` 和 `compareAndSetState()` (CAS操作) 来管理这个状态。
 2. CLH队列: 一个虚拟的、双向的FIFO队列 (CLH队列的变体), 用于存放所有等待获取锁的线程。当线程获取锁失败时, 会被封装成一个 `Node` 节点并加入到队列的尾部。

2. AQS是如何实现独占模式和共享模式的？★★★★★

AQS通过让子类重写不同的模板方法来支持两种模式：

- **独占模式 (Exclusive Mode)**：同一时间只允许一个线程持有锁。
 - 代表：`ReentrantLock`。
 - 需重写的方法：`tryAcquire(int arg)` 和 `tryRelease(int arg)`。
- **共享模式 (Shared Mode)**：同一时间允许多个线程持有锁。
 - 代表：`Semaphore`, `CountDownLatch`, `ReentrantReadWriteLock`。
 - 需重写的方法：`tryAcquireShared(int arg)` 和 `tryReleaseShared(int arg)`。

AQS的 `acquire / release` 等公共方法会调用这些由子类实现的模板方法来执行具体的同步逻辑。

3. AQS是如何实现线程的等待和唤醒的？★★★★☆

AQS不使用 `Object.wait()` 和 `notify()`，而是依赖于 `java.util.concurrent.locks.LockSupport` 这个工具类：

- **等待 (阻塞)**：当线程获取锁失败并加入等待队列后，AQS会调用 `LockSupport.park(this)` 来将当前线程挂起，使其进入 `WAITING` 状态。这个操作非常高效，因为它直接利用了操作系统层面的线程调度。
- **唤醒**：当一个线程释放锁后，它会找到等待队列中的头节点（的下一个节点），并调用 `LockSupport.unpark(thread)` 来唤醒这个等待中的线程，使其从 `park` 方法返回，并再次尝试获取锁。

4. AQS的同步队列和条件队列原理？★★★★★

AQS内部维护了两类队列：

- **同步队列 (Sync Queue)**：
 - **结构**：就是前述的CLH双向队列。
 - **作用**：所有尝试获取锁失败的线程都会被放入这个队列中排队等待。
 - **特点**：队列中的节点与线程一一对应，遵循FIFO原则（在公平模式下）。当锁被释放时，会从队列头部唤醒一个节点。
- **条件队列 (Condition Queue)**：
 - **结构**：是一个单向链表，与一个 `Condition` 对象关联（通过 `lock.newCondition()` 创建）。
 - **作用**：用于实现线程间的通信（类似于 `Object.wait/notify`）。
 - **原理**：
 1. 一个已经获取到锁的线程调用 `condition.await()` 方法。
 2. 该线程会释放持有的锁，并被封装成一个节点，加入到这个 `Condition` 对象对应的条件队列尾部。
 3. 当另一个线程调用 `condition.signal()` 或 `signalAll()` 时，会从条件队列的头部取出一个节点，并将其移动到同步队列的尾部。
 4. 被移动的线程被唤醒后，会像其他线程一样，重新开始竞争同步队列中的锁。

5. AQS为什么采用双向链表？★★★★☆

采用双向链表的设计是出于功能和性能的综合考虑：

1. **节点自我删除**：当一个等待中的线程因为超时或被中断而需要放弃等待时，它需要将自己从队列中移除。双向链表使得一个节点可以轻松地找到它的前驱（`prev`）和后继（`next`）节点，然后通过修改指针（`prev.next = next; next.prev = prev;`）将自己安全地“摘除”，而不会打断整个队列。
2. **避免遍历**：在唤醒后继节点时，或者在判断是否需要挂起当前线程时，节点需要知道其前驱节点的状态。双向链表提供了对前驱节点的 $O(1)$ 复杂度的访问，避免了从头遍历。

8.2 锁的分类与特性

1. 公平锁和非公平锁有什么区别？ReentrantLock默认是哪种？为什么会有这种设计？★★★★☆

- **区别**：
 - **公平锁**：遵循**先来后到（FIFO）**的原则。等待时间最长的线程会最先获取到锁。
 - **非公平锁**：允许“插队”。当锁被释放时，新来的线程和队列中等待的线程会一起竞争，新来的线程可能先于等待已久的线程获取到锁。
- **ReentrantLock 默认**：默认是**非公平锁**。可以在构造时传入 `true` 来创建公平锁（`new ReentrantLock(true)`）。
- **设计原因**：为了**更高的吞吐量**。在非公平模式下，当一个线程释放锁时，如果恰好有一个新线程请求锁，它可能可以直接获取成功而无需被挂起。这避免了一次线程唤醒和上下文切换的开销，从而提升了系统的整体性能。公平锁为了保证顺序，则必须承担这些额外的开销。

2. 什么是读写锁（ReentrantReadWriteLock）？它解决了什么问题？★★★★☆

- **是什么**：读写锁是一种特殊的锁，它将锁的功能分为**读锁**和**写锁**。
- **解决了什么问题**：它解决了在**读多写少**的并发场景下，使用普通互斥锁（如 `ReentrantLock`）导致并发性能低下的问题。
- **规则**：
 - **读锁是共享的**：多个线程可以同时持有读锁，互不影响。
 - **写锁是独占的**：一次只能有一个线程持有写锁。
 - **读写互斥**：当有线程持有写锁时，其他线程不能获取读锁或写锁。当有线程持有读锁时，其他线程不能获取写锁。

3. ReentrantReadWriteLock的读锁和写锁是如何实现互斥和共享的？★★★★★

`ReentrantReadWriteLock` 的实现同样基于AQS。它巧妙地利用AQS的 `state` 变量来同时管理两种锁的状态。

- **读锁（共享模式）**：当线程请求读锁时，会以**共享模式**调用AQS的 `acquireShared()`。只要当前没有写锁，AQS就会允许该线程获取成功，并将 `state` 值增加。多个线程可以同时成功。
- **写锁（独占模式）**：当线程请求写锁时，会以**独占模式**调用AQS的 `acquire()`。只有当 `state` 为0（即没有任何读锁或写锁）时，它才能获取成功。
- **互斥与共享**：AQS的 `state` 值被拆分为两部分，高16位表示读锁的数量，低16位表示写锁的重入次数。通过对 `state` 进行位运算，就可以在一次原子操作中判断出当前的锁状态，从而实现读写锁的复杂逻辑。

4. 读写锁的state变量是如何同时表示读锁和写锁状态的? ★★★★★

`ReentrantReadWriteLock` 使用一个32位的 `int` 类型的 `state` 变量, 通过按位切分的方式来同时表示两种锁的状态:

- **高16位**: 用来记录读锁的持有数量。当一个线程获取读锁时, 高16位的值会加1。
- **低16位**: 用来记录写锁的重入次数。当一个线程获取写锁时, 低16位的值会加1。

判断逻辑:

- `state != 0` 时, 如果低16位为0, 说明当前只有读锁; 如果高16位为0, 说明当前只有写锁。
- 获取写锁的条件是 `state` 必须为0 (或者当前线程已持有写锁, 进行重入)。
- 获取读锁的条件是低16位必须为0 (即没有任何写锁)。

5. 什么是可重入锁, 怎么实现可重入锁? ★★★★★☆

- **是什么**: 可重入锁 (Reentrant Lock), 也叫递归锁, 指的是同一个线程可以多次获取同一把锁, 而不会造成死锁。
- **怎么实现**: 锁内部维护一个计数器和一个持有锁的线程的引用。
 1. 当一个线程请求锁时, 如果锁未被持有, 则该线程获取锁, 计数器置为1。
 2. 如果该线程再次请求锁, 则只需将计数器加1。
 3. 当线程释放锁时, 计数器减1。
 4. 只有当计数器减到0时, 该锁才会被真正释放, 其他线程才有机会获取。
- **例子**: `synchronized` 和 `ReentrantLock` 都是可重入锁。

6. 如何实现无锁化编程? ★★★☆☆

无锁化编程是一种不使用锁 (如 `synchronized`, `Lock`) 来实现线程安全的技术, 它可以有效避免死锁和减少线程上下文切换的开销。主要有以下几种方式:

1. **CAS (比较并交换)**: 利用CPU提供的原子指令, 以乐观锁的方式更新数据。JUC包下的原子类 (`AtomicInteger` 等) 是其典型应用。
2. **ThreadLocal (线程本地存储)**: 为每个线程提供一个独立的变量副本, 从根本上避免了线程间的数据共享和竞争。
3. **不可变对象 (Immutability)**: 使用 `final` 关键字创建的对象, 其状态在创建后无法被修改, 因此天生就是线程安全的, 可以被自由共享。

9. ThreadLocal

1. ThreadLocal的作用和工作原理是什么? 它与同步机制有什么根本区别?

★★★★☆

- **作用**: `ThreadLocal` 提供了一种线程本地变量。它为使用该变量的每个线程都提供一个独立的变量副本, 从而做到了线程隔离。每个线程都只能访问自己的副本, 不会影响其他线程的副本。
- **工作原理**:
 1. 每个 `Thread` 对象内部都有一个 `ThreadLocalMap` 类型的成员变量 `threadLocals`。
 2. `ThreadLocalMap` 是一个定制化的哈希表, 其 **Key** 是 `ThreadLocal` 对象本身的弱引用, **Value** 则是该线程的变量副本。
 3. 当调用 `threadLocal.set(value)` 时, 实际上是获取当前线程的 `ThreadLocalMap`, 然后以 `threadLocal` 对象为键, `value` 为值, 存入 `Map` 中。

4. 调用 `threadLocal.get()` 时，也是先获取当前线程的 `ThreadLocalMap`，再以 `threadLocal` 为键，从中取出对应的值。

- **与同步机制的根本区别：**

- **同步机制 (synchronized, Lock)：**是一种“以时间换空间”的策略。它通过加锁，让多个线程排队访问同一个共享变量，保证了数据的一致性，但牺牲了并发性。
- **ThreadLocal：**是一种“以空间换时间”的策略。它不加锁，而是为每个线程都创建一个独立的变量副本，从而避免了线程间的竞争。它不解决共享变量的并发问题，而是从根本上避免了共享。

2. 什么是ThreadLocal，如何实现的？★★★★☆

(此题答案与上一题的核心内容重合，可参考上一题的“作用”和“工作原理”部分进行回答。)

3. ThreadLocal有哪些常见的应用场景？举例说明。★★★★☆

1. **管理数据库连接：**在多线程环境下，为每个线程分配一个独立的数据库连接，避免了连接的频繁创建和关闭，也避免了线程间的连接竞争。`MyBatis` 等持久层框架就使用了这种机制。
2. **管理用户会话 (Session)：**在Web应用中，一个请求通常由一个线程处理。使用 `ThreadLocal` 可以方便地在处理该请求的整个调用链中（从Controller到Service再到DAO），随时获取当前用户的会话信息，而无需在每个方法间层层传递。
3. **存储事务上下文：**在声明式事务管理中（如Spring），`ThreadLocal` 被用来存储当前线程的事务状态，确保同一线程中的所有数据库操作都在同一个事务中。

4. ThreadLocal为什么会引起内存泄漏问题？如何避免？★★★★★

- **内存泄漏原因：**

1. `ThreadLocalMap` 的 **Key** (即 `ThreadLocal` 对象) 是一个**弱引用 (WeakReference)**。当外部不再再有对 `ThreadLocal` 对象的强引用时，GC会回收它，此时 `ThreadLocalMap` 中会出现 **Key为 null 的Entry**。
2. 但是，这个Entry的 **Value** 仍然被 `ThreadLocalMap` **强引用着**。
3. 如果这个线程是一个**线程池中的复用线程**，它会一直存活，其 `ThreadLocalMap` 也一直存在。那么，这个Key为 `null` 的Entry及其强引用的Value就永远不会被回收，从而导致了**内存泄漏**。

- **如何避免：**

- **最佳实践：**在每次使用完 `ThreadLocal` 后，务必在 `finally` 块中调用其 `remove()` 方法。`remove()` 方法会从当前线程的 `ThreadLocalMap` 中移除这个Entry，从而释放对Value的强引用。

5. ThreadLocalMap中的键和值分别是什么引用类型？为什么这样设计会导致内存泄漏？★★★★★

- **键 (Key)：**是 `ThreadLocal` 对象的**弱引用 (WeakReference)**。
- **值 (Value)：**是**强引用 (Strong Reference)**。
- **为什么会发生内存泄漏：**
 - **设计初衷：**将Key设计为弱引用，是为了当外部的 `ThreadLocal` 对象被回收时，`ThreadLocalMap` 中的对应条目可以在下一次 Map 操作时被自动清理掉，这在一定程度上能防止内存泄漏。

- **泄漏发生**：但问题在于，如果一个线程执行完任务后，没有调用 `threadLocal.remove()`，并且这个线程被线程池回收复用，那么这个线程就一直存活。虽然GC回收了 `ThreadLocal` 对象使Key变成了 `null`，但Value仍然被 `ThreadLocalMap` 强引用。只要这个线程不消亡，这个Value就永远无法被回收，内存泄漏就发生了。

6. 为什么InheritableThreadLocal为啥还需要TransmittableThreadLocal?

★★★★☆

- **InheritableThreadLocal (ITL) 的局限性**：ITL 只能在**创建子线程**的那一刻，将父线程中的本地变量复制一份给子线程。它无法解决在使用**线程池**时的上下文传递问题。因为线程池会复用线程，任务提交给线程池时，执行任务的线程可能是一个已经存在的旧线程，而不是新创建的子线程，ITL 的传递机制会失效。
- **TransmittableThreadLocal (TTL) 的出现**：TTL 是阿里巴巴开源的一个库，专门为了解决上述问题。
 - **原理**：它通过**装饰 (Decorate)** `Runnable` 和 `Callable` 接口，在任务**提交给线程池时**，将父线程的 `ThreadLocal` 上下文捕获下来。在任务**即将被线程池中的线程执行前**，再将捕获的上下文设置到执行线程中。任务执行完毕后，它还会负责清理上下文。
 - **结论**：TTL 解决了在使用线程池等会涉及线程复用的场景下，实现可靠的父子线程（或任务提交方与任务执行方）之间的上下文传递问题。

7. 线程池中使用ThreadLocal会有哪些潜在的风险？★★★★☆

主要风险是**数据污染**和**内存泄漏**。

- **数据污染**：线程池会复用线程。如果上一个任务在使用完 `ThreadLocal` 后没有调用 `remove()` 清理，那么下一个任务由同一个线程执行时，会读取到上一个任务遗留下的脏数据，导致严重的业务逻辑错误。
- **内存泄漏**：原因同问题4和问题5。线程池中的线程通常是长期存活的，如果 `ThreadLocal` 变量不被移除，其Value对象会一直占用内存，最终可能导致OOM。

10. 其他并发问题

1. 什么是竞态条件 (Race Condition)？如何避免？★★★☆☆

- **是什么**：当多个线程访问和操作同一个共享数据，并且最终的执行结果取决于这些线程的执行顺序时，就发生了竞态条件。
- **如何避免**：核心是保证对共享数据操作的**原子性**。
 1. 使用**锁**（`synchronized` 或 `Lock`）来保护访问共享数据的代码块，确保同一时间只有一个线程能操作它。
 2. 使用**原子类**（如 `AtomicInteger`）来代替普通变量进行原子操作。

2. 什么是死锁 (Deadlock)？发生死锁的四个必要条件是什么？如何避免死锁？

★★★★★

- **是什么**：死锁是指两个或多个线程在执行过程中，因争夺资源而造成的一种互相等待的现象，若无外力干涉，它们都将无法推进下去。
- **四个必要条件**（必须同时满足）：
 1. 互斥条件 (Mutual Exclusion)：资源在同一时间只能被一个线程持有。

- 请求与保持条件 (Hold and Wait) : 线程在持有至少一个资源的同时, 又去请求其他线程正持有的资源。
- 2. **不剥夺条件 (No Preemption)** : 线程已获得的资源, 在未使用完之前, 不能被强行剥夺。
- 3. **循环等待条件 (Circular Wait)** : 存在一个线程—资源的循环等待链。
- **如何避免**: 破坏上述四个必要条件中的任何一个。
 - 1. **破坏请求与保持**: 一次性申请所有需要的资源。
 - 2. **破坏不剥夺**: 如果获取不到新资源, 主动释放已持有的资源。
 - 3. **破坏循环等待**: 按固定的顺序来申请资源, 避免形成环路。

3. 什么是活锁 (Livelock) 和线程饥饿 (Thread Starvation) ? 如何避免?

★★★★☆

- **活锁 (Livelock)** : 指线程们都在积极地响应对方, 但却导致整体上没有任何进展。就像两个人狭路相逢, 都想给对方让路, 结果你向左我向右, 你向右我向左, 一直在动但就是过不去。
 - **避免**: 引入随机性, 例如在重试逻辑中加入随机的等待时间。
- **线程饥饿 (Thread Starvation)** : 指某个线程由于一直无法获得CPU时间片或所需的资源, 而导致长时间无法执行。
 - **原因**: 可能因为线程优先级过低, 或者在高并发的非公平锁中, 一个线程总是“运气不好”抢不到锁。
 - **避免**: 设置合理的线程优先级、使用公平锁、保证资源充足。

4. 如何保证多线程下i++结果正确? ★★★★★

`i++` 不是一个原子操作, 它包含“读-改-写”三步。要保证其在多线程下的正确性, 必须保证这三步的原子性。

1. **使用 synchronized 关键字**: 将 `i++` 操作放在同步代码块中。
2. **使用 Lock**: 使用 `ReentrantLock` 等显式锁来保护 `i++` 操作。
3. **使用原子类 (推荐)**: 使用 `java.util.concurrent.atomic.AtomicInteger`, 调用其 `incrementAndGet()` 或 `getAndIncrement()` 方法, 这是最高效、最方便的方式。

5. 线程池中LongAdder和AtomicLong的区别? ★★★★★

`AtomicLong` 和 `LongAdder` 都用于实现原子性的长整型计数器, 但在高并发场景下, `LongAdder` 的性能远超 `AtomicLong`。

- `AtomicLong`: 内部只有一个 `value` 字段, 所有线程都通过CAS操作来竞争更新这同一个 `value`。在高并发下, CAS失败和自旋的概率会急剧增加, 导致性能瓶颈。
- `LongAdder (JDK 1.8+)`: 内部采用**分段计数**的思想。它维护一个 `base` 值和一个 `cell` 数组。在无竞争时, 直接CAS更新 `base` 值。当出现竞争时, 会将不同线程的计数请求分散到不同的 `cell` 上, 每个线程只更新自己的 `cell`。最后获取总数时, 再将 `base` 和所有 `cell` 的值相加。通过**空间换时间**, 极大地减少了热点竞争。

6. 有三个线程T1,T2,T3如何保证顺序执行? ★★★★★

使用 `Thread.join()` 方法。 `join()` 方法会使当前线程等待调用该方法的线程执行完毕。

```

Thread t1 = new Thread(() -> System.out.println("T1"));
Thread t2 = new Thread(() -> System.out.println("T2"));
Thread t3 = new Thread(() -> System.out.println("T3"));

t1.start();
t1.join(); // 主线程等待t1执行完毕

t2.start();
t2.join(); // 主线程等待t2执行完毕

t3.start();

```

7. 三个线程分别顺序打印0-100? ★★★★★

这道题考察的是线程间的精确通信。可以使用 `synchronized` 配合 `wait()` 和 `notifyAll()` 实现。

```

public class SequentialPrint {
    private int count = 0;
    private final Object lock = new Object();

    public void print() {
        new Thread(() -> {
            while (count <= 100) {
                synchronized (lock) {
                    if (count % 3 == 0) {
                        System.out.println("Thread 1: " + count++);
                        lock.notifyAll();
                    } else {
                        try {
                            lock.wait();
                        } catch (InterruptedException e) {
                            e.printStackTrace();
                        }
                    }
                }
            }
        }).start();
        // 另外两个线程类似，只需修改if判断条件为 (count % 3 == 1) 和 (count % 3 == 2)
    }
}

```

8. 为什么不能在try-catch中捕获子线程的异常? ★★☆☆☆

因为**线程是独立的执行单元**，**每个线程都有自己独立的执行栈**。当一个子线程抛出未捕获的异常时，该异常是沿着**子线程自己的调用栈**向上传播的，它不会“跨越”到父线程的执行栈中。因此，父线程的 `try-catch` 块根本无法捕获到子线程的异常。

要处理子线程的异常，可以：

1. 在子线程的 `run()` 方法内部进行 `try-catch`。
2. 使用 `Thread.setUncaughtExceptionHandler()` 为线程设置一个未捕获异常处理器。

9. Java线程出现异常，进程为啥不会退出？ ★★★☆☆

在一个Java进程中，一个线程因为未捕获的异常而终止，只会结束**该线程自己**的生命周期。

- 只要进程中还存在**任何一个非守护线程 (non-daemon thread)** 在运行，JVM进程就不会退出。
- `main` 方法所在的线程就是一个非守护线程。只有当所有非守护线程都执行完毕后，JVM才会退出。

10. 什么是虚拟线程，JDK 21中的虚拟线程是怎么回事？ ★★★★★

- **是什么：**虚拟线程 (Virtual Threads) 是JDK 21正式引入的一项重大特性 (源自Project Loom) 。它是一种极其**轻量级**的线程实现，由JVM自身进行管理和调度。
- **怎么回事：**
 1. 传统的Java线程 (平台线程) 与操作系统内核线程是一一对应的，创建成本高，数量有限。
 2. 虚拟线程不直接映射到内核线程。JVM可以将**成千上万个虚拟线程**调度运行在**少数几个平台线程**之上。
 3. 当一个虚拟线程执行阻塞I/O操作时，它不会阻塞底层的平台线程，而是会被JVM“卸载”，让平台线程去执行其他虚拟线程。当I/O操作完成后，JVM会再将这个虚拟线程“挂载”回某个平台线程继续执行。
 4. 这使得我们可以用传统的“一个请求一个线程”的同步阻塞式编程模型，来编写出具有极高吞吐量的、类似异步非阻塞风格的应用程序。

11. 为什么虚拟线程不能用synchronized？ ★★★☆☆

更准确的说法是“**应谨慎使用 `synchronized`**”。

- 当一个虚拟线程进入 `synchronized` 代码块并被阻塞时，它会“**钉住**” (pin) 其当前所在的平台线程。
- 这意味着这个平台线程无法被JVM用来执行其他虚拟线程，它会像传统线程一样被阻塞。
- 如果大量虚拟线程都使用 `synchronized` 并发生阻塞，就会耗尽底层平台线程池的线程，从而严重削弱虚拟线程带来的高吞吐量优势。
- **替代方案：**推荐使用 `java.util.concurrent.locks.ReentrantLock`，因为它已经被适配为在使用虚拟线程时不会钉住平台线程。

12. 为什么虚拟线程不要和线程池一起用？ ★★★☆☆

因为**虚拟线程本身就是为了替代线程池而设计的**。

- 线程池的核心目的是**复用昂贵的平台线程**。
- 而虚拟线程的创建成本极低，它们就是“一次性”的。最佳实践是**为每一个任务都创建一个新的虚拟线程** (`Thread.startVirtualThread(task)`)。
- 如果将虚拟线程放入池中进行复用，不仅毫无意义，反而增加了不必要的复杂性，完全违背了虚拟线程的设计初衷。

13. 为什么虚拟线程尽量避免使用ThreadLocal？ ★★★☆☆

因为**虚拟线程的数量可能非常巨大**。

- `ThreadLocal` 为每个使用它的线程都创建一个数据副本。
- 如果你创建了数百万个虚拟线程，并且每个线程都使用了 `ThreadLocal`，那么就会创建数百万个数据副本，这可能导致**巨大的内存消耗**，甚至引发OOM。

- 在虚拟线程的场景下，需要重新审视数据传递和共享的方式，例如通过方法参数来传递上下文，而不是依赖 `ThreadLocal`。

14. 如何在Java中实现高效的异步编程？如何避免回调地狱？★★★★☆

- **如何实现：**现代Java中实现高效异步编程的最佳方式是使用 `CompletableFuture` (JDK 1.8+)。
 - `CompletableFuture` 提供了一套丰富的API，可以轻松地创建异步任务（`supplyAsync`，`runAsync`），并以非阻塞的方式处理其结果。
- **如何避免回调地狱（Callback Hell）：**`CompletableFuture` 通过其**链式调用**和**组合**的能力，完美地解决了回调地狱问题。
 - **链式调用：**可以使用 `thenApply()`（转换结果）、`thenAccept()`（消费结果）、`thenRun()`（执行后续动作）等方法，将一系列异步操作优雅地链接起来，形成一个清晰的、顺序的逻辑流。
 - **组合：**可以使用 `thenCombine()`（组合两个独立CF的结果）、`allOf()`（等待所有CF完成）、`anyOf()`（等待任意一个CF完成）等方法，来组合多个独立的异步任务。

通过 `CompletableFuture`，可以将原本嵌套的、难以理解的回调代码，改写成线性的、声明式的、易于阅读和维护的代码。